

Product Requirements Document

Self-Serve Product Analytics Dashboard

Author: Ashish Bajpayee

Status: Draft — v1.0

Last Updated: July 2026

Stakeholders: Product, Growth, Engineering, Data/Analytics, Design

1. Overview

Product and Growth teams currently rely on fragmented reports, manual data pulls, and ad-hoc SQL queries to understand feature adoption, retention, and experiment performance. This slows down decision-making, creates inconsistent metric definitions across teams, and delays roadmap prioritization.

The Self-Serve Product Analytics Dashboard is an internal platform that lets Product Managers, Growth PMs, and stakeholders explore product usage data — funnels, retention cohorts, feature adoption, and A/B test results — without depending on data analysts or engineering for every query.

2. Problem Statement

- Product and Growth teams rely on fragmented reports and manual analysis to measure feature adoption and retention.
- Getting a simple funnel or cohort view requires filing a request with a data analyst or writing custom SQL, adding days of turnaround time.
- Metric definitions (e.g., "active user," "adoption") are inconsistently calculated across teams, leading to conflicting numbers in roadmap discussions.
- Experimentation results are scattered across spreadsheets, making it hard to track which A/B tests shipped, their outcomes, and follow-up actions.
- This results in slower experimentation cycles and delayed, less-informed roadmap prioritization.

3. Goals & Success Metrics

Product Goals

- Enable any PM or Growth stakeholder to answer common product questions (adoption, retention, funnel drop-off) without engineering/analyst help.
- Establish a single source of truth for core product metrics.
- Reduce the time from "question" to "insight" from days to minutes.

Success Metrics (North Star + Supporting KPIs)

Metric	Baseline	Target (90 days post-launch)
Time to insight (question → answer)	2–3 days (analyst request)	< 10 minutes (self-serve)

Metric	Baseline	Target (90 days post-launch)
% of ad-hoc analyst data requests related to product metrics	100%	↓ 60%
Weekly active users of the dashboard (PMs/Growth)	0	70% of PM/Growth org
Number of experiments tracked in-platform	0 (scattered in sheets)	100% of active experiments
Data trust score ("I trust this data")	N/A	≥ 8/10 average

North Star Metric: Weekly Active Insight Generation — the number of distinct analyses (funnels, cohorts, adoption views) run by non-analyst users per week.

4. Target Users & Personas

Persona	Needs
Product Manager	Track feature adoption, understand drop-off in onboarding funnels, prioritize backlog using usage data
Growth PM	Monitor retention cohorts, evaluate experiment lift, identify growth loops
Engineering Lead	Validate technical rollout impact (e.g., did the new API reduce churn)
Leadership / Stakeholders	View high-level KPI trends without needing raw data access

5. Scope

In Scope (MVP)

- Funnel Analytics — define and visualize multi-step funnels (e.g., signup → activation → first key action).
- Retention Cohorts — cohort retention curves (D1/D7/D30, weekly/monthly cohorts).
- Feature Adoption Tracking — % of eligible users who adopted a given feature over time, segmented by plan/role/geography.
- Experimentation Reporting — view active/completed A/B tests, variant performance, and statistical significance.
- Self-serve filtering — segment any view by user attributes (plan tier, signup date, region, device).
- Saved views & sharing — save a dashboard configuration and share a link with a team.

Out of Scope (Future Phases)

- Predictive / ML-based churn scoring
- Real-time (sub-minute) streaming dashboards
- Fully custom SQL query builder for non-technical users (candidate for post-MVP "Advanced Mode")
- Automated anomaly detection / alerting

6. User Stories

Funnel Analytics

- As a PM, I want to define a custom funnel (e.g., Sign Up → Onboarding Complete → First Purchase) so I can see where users drop off.
- As a PM, I want to compare funnel conversion across segments (new vs. returning users) so I can identify at-risk cohorts.

Retention Cohorts

- As a Growth PM, I want to view D1/D7/D30 retention by signup cohort so I can measure onboarding effectiveness over time.
- As a Growth PM, I want to compare retention curves before and after a feature launch to measure impact.

Feature Adoption

- As a PM, I want to see what % of eligible users have adopted a new feature within 30/60/90 days so I can judge feature success against goals.

Experimentation

- As a PM, I want to see all active experiments with sample size, lift, and confidence level so I can make ship/no-ship decisions without pinging the data team.

Platform

- As any user, I want to save and share a dashboard view via link so my team sees the same numbers I do.

7. Functional Requirements

#	Requirement	Priority
1	Users can build funnels from a list of tracked events (drag/drop or dropdown-based, no code)	Must
2	Retention cohort view with configurable time windows	Must
3	Feature adoption view with denominator = eligible user base (configurable)	Must
4	Segment/filter any report by user property (plan, region, signup date, device)	Must
5	Experiment results view pulling from the experimentation platform (variant, lift, confidence)	Must
6	Save & share dashboard views via URL	Should
7	Export view as CSV/PNG	Should
8	Role-based access control (view-only vs. edit)	Should
9	Central metric glossary/definitions surfaced in-app	Should
10	Alerting on metric threshold breach	Could (Phase 2)
11	Natural-language query interface ("show retention for enterprise users")	Won't (MVP)

8. Prioritization (RICE)

Feature	Reach	Impact	Confidence	Effort	Priority
Funnel Analytics	High	High	High	Medium	P0
Retention Cohorts	High	High	High	Medium	P0
Feature Adoption Tracking	Medium	High	High	Low	P0
Experimentation Reporting	Medium	Medium	Medium	Medium	P1
Saved Views / Sharing	High	Medium	High	Low	P1
Metric Glossary	Medium	Medium	High	Low	P1
RBAC	Medium	Low	High	Low	P2

9. Success Metrics per Feature

Feature	Metric
Funnel Analytics	# of funnels created/week; avg. time to build a funnel
Retention Cohorts	# of cohort views generated/week
Feature Adoption	# of features tracked; adoption view usage rate
Experimentation Reporting	% of live experiments visible in-platform
Overall Platform	WAU of dashboard among PM/Growth org; reduction in ad-hoc data requests

10. How to Build It — Technical Approach

This section reflects how the MVP was actually built (Python + SQL) and how to extend it into a production-grade platform.

10.1 Architecture Overview

- Event Sources (App / Web / Backend) generate raw usage events.
- Event Ingestion Layer — SDK or API pushes events into a queue (e.g., Kafka).
- Raw Event Store — a data warehouse (PostgreSQL / Snowflake / BigQuery) persists all events.
- Transformation Layer — dbt or Python ETL jobs build derived tables: funnels, cohorts, adoption, experiment results.
- Metrics / Semantic Layer — defines "active user," "adoption," etc. once, centrally, so every team sees the same number.
- Backend API (Python — FastAPI/Flask) serves pre-aggregated and on-demand queries.
- Frontend Dashboard (React + Recharts/D3) renders filterable, shareable views for PMs and Growth stakeholders.

10.2 MVP Build Plan (matches resume project scope)

1. Event Tracking Audit

- Inventory existing tracked events (signup, login, feature_used, purchase, etc.).
- Identify gaps — instrument missing events with the product analytics SDK (Mixpanel/Segment) or custom event logging.

2. Data Pipeline (Python + SQL)

- Use Python scripts (or scheduled Airflow/cron jobs) to pull raw events from the source system into a warehouse table.
- Write SQL transformations to pre-compute: funnel_steps, retention_cohorts, feature_adoption, and experiment_results tables.

3. Metrics Layer

- Centralize metric definitions in one config (YAML/SQL model) so "retention" or "active user" is defined once and reused everywhere.

4. Backend API

- Build lightweight REST endpoints (Python, FastAPI) that query pre-aggregated tables based on filters (date range, segment, cohort).
- Cache frequent queries (Redis) to keep dashboard load times low.

5. Frontend Dashboard

- Build filterable chart components: funnel visualization, cohort retention heatmap/curve, adoption trend line, experiment comparison table.
- Add save/share functionality (persist filter state to a shareable URL or saved-view ID).

6. Access & Rollout

- Start with a pilot group (one Product + one Growth team).
- Gather feedback on metric trust and usability before wider rollout.
- Add RBAC before opening access company-wide.

10.3 Tech Stack Suggestion

Layer	Tool
Event tracking	Segment / Mixpanel / custom SDK
Data warehouse	PostgreSQL / Snowflake / BigQuery
Transformation	Python (pandas) + SQL, or dbt for scale
Backend API	Python (FastAPI)
Caching	Redis
Frontend	React + Recharts/D3
Experimentation source	Existing A/B testing platform (Optimizely/Statsig/in-house)

10.4 Key Technical Risks

- Data volume: Naively querying raw events for every dashboard load won't scale — pre-aggregation is essential.
- Metric consistency: Without a single semantic layer, teams will keep computing "retention" differently — must be solved early, not patched later.

- Event schema drift: Product/eng changes to event names or properties can silently break funnels/cohorts — needs schema versioning and alerts.

11. Rollout Plan

Phase	Scope	Timeline
Phase 0	Event audit, metrics definitions agreed with stakeholders	Weeks 1–2
Phase 1 (MVP)	Funnel + retention + adoption views, pilot with 1–2 teams	Weeks 3–8
Phase 2	Experimentation reporting, saved views/sharing	Weeks 9–12
Phase 3	Company-wide rollout, RBAC, metric glossary	Weeks 13–16
Phase 4 (Future)	Alerting, NL query interface, predictive analytics	Post-launch

12. Risks & Mitigations

Risk	Mitigation
Low adoption if UI is too complex for non-technical PMs	Run usability testing with PMs during MVP; prioritize simplicity over feature count
Data trust issues if numbers don't match existing reports	Reconcile against known "gold standard" reports before wide rollout
Performance issues at scale	Pre-aggregate data; add caching layer early
Scope creep toward a full BI tool	Explicitly cap MVP scope; defer advanced features to later phases

13. Open Questions

- Which existing analytics tool (if any) should this replace vs. complement?
- What is the authoritative source of truth for user segmentation attributes (plan, region)?
- Who owns metric definition governance long-term — Product, Data, or a shared council?

This PRD is written as a portfolio case study based on a self-serve analytics MVP built using Python and SQL to enable real-time product performance monitoring for Product and Growth teams.